S7: Introduction to Computer Vision with CNNs, Transfer Learning, and Fine-tuning

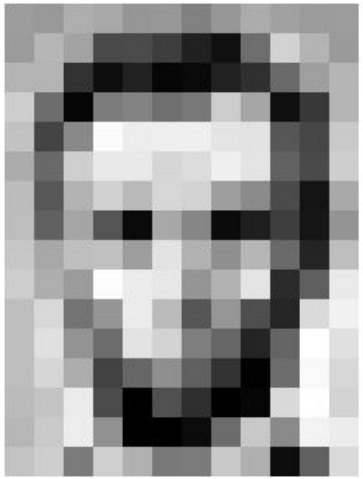
By Phanuphat (Oad) Srisukhawasu

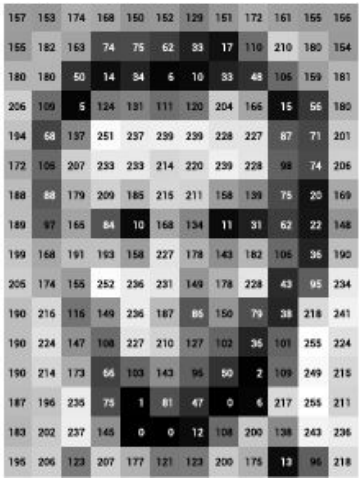
Machine Learning Department, NUS Fintech Society

Computer Vision - Definition

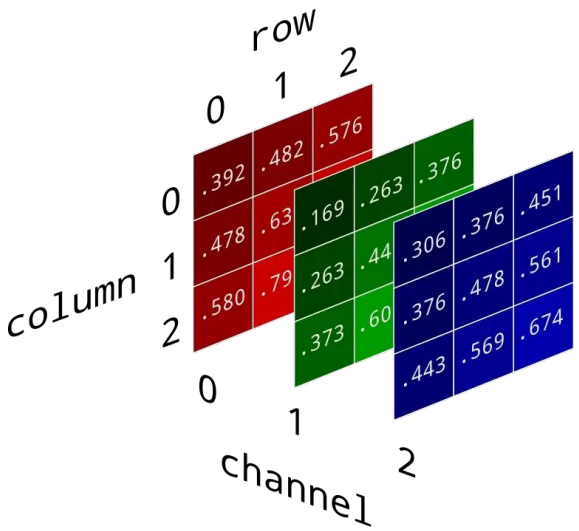
Computer vision is a field of artificial intelligence (AI) enabling computers to derive information from images (+ other media.)

Our main focus is on "image" data! ⇒ ***2D/3D Array of Numbers***





Images are Just 2D/3D Array of Numbers



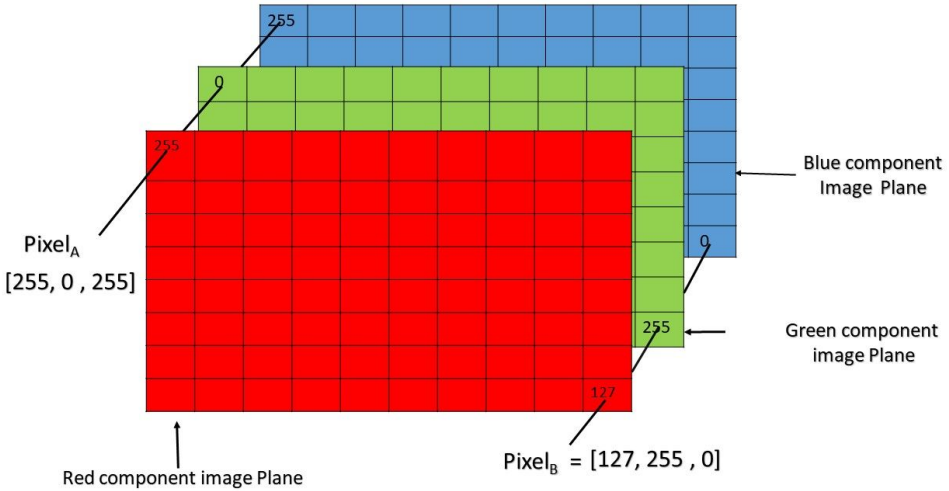
Grayscale Images ⇒ 2D (*H x W*)

Color Images ⇒ (*H x W x C*)

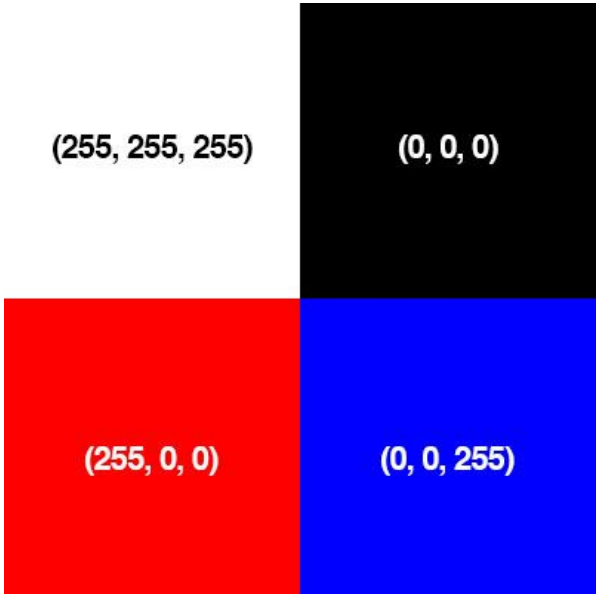
H is the height
W is the width
C is the color channels (RGB)

The number is the **pixel brightness (0 - 255).**

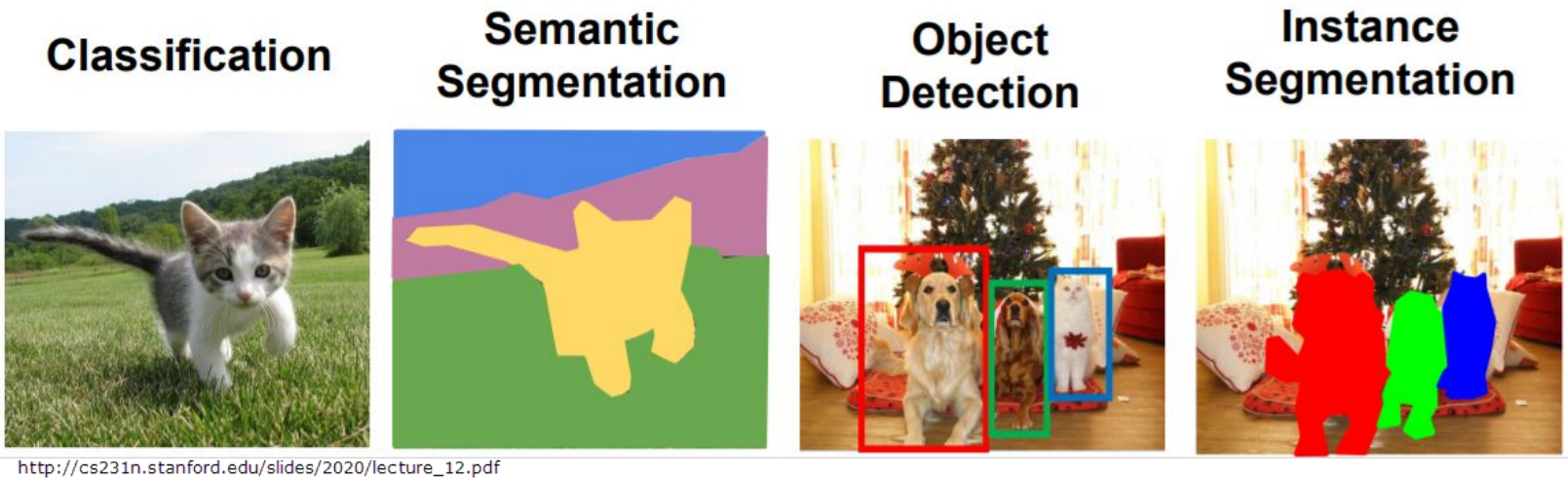
Pixel Brightness



Pixel of an RGB image are formed from the corresponding pixel of the three component images



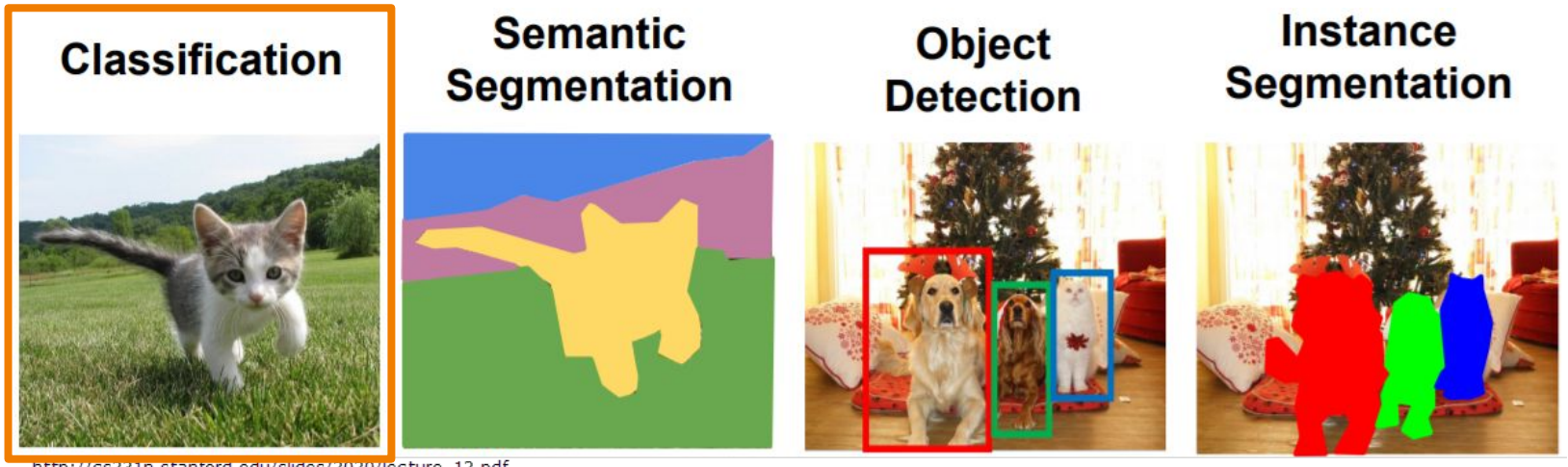
Tasks in Computer Vision (For Images)



http://cs231n.stanford.edu/slides/2020/lecture_12.pdf

Our Main Focus for Today

Regression + Classification + Encoder-Decoder



http://cs231n.stanford.edu/slides/2020/lecture_12.pdf

Image Classification in TensorFlow

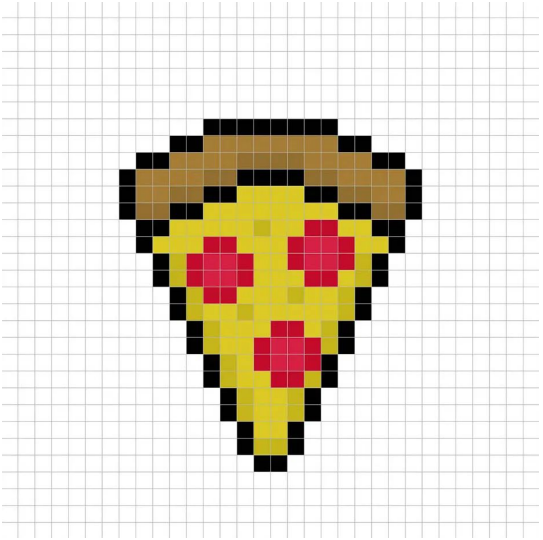
There is a really long history of how image classification has evolved until now. We will skip ahead to the **modern era** where **neural networks** are already viable.

- **Sequential data**: RNNs, GRUs, LSTMs (Previous lessons) which have potentials for looking back and forth.
- **Spatial data**: Convolutional Neural Networks (CNNs) *which have potentials for looking at the surroundings*.

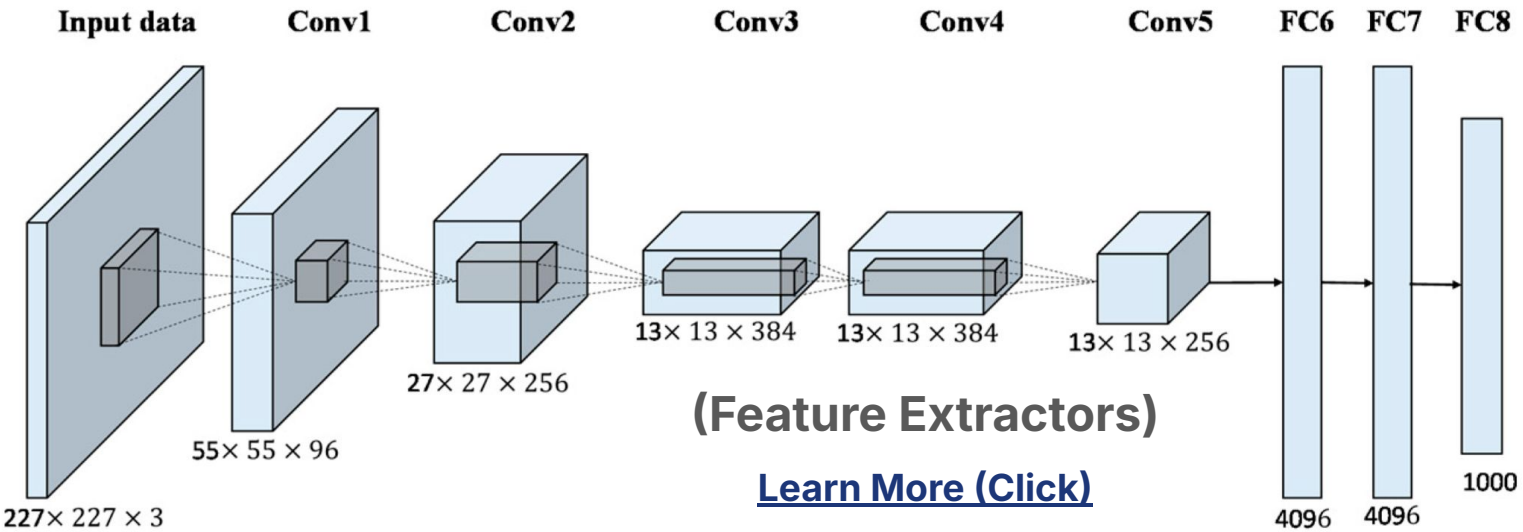
Sequential vs Spatial Data

Text and **time series** are example of **sequential** data: each instance is related to the instances that come before and after it.

Images are spatial data: each pixel is related to the pixels surrounding it (area-wise POV) → **Larger in Size = Better in Resolution**



CNNs Work Really Well on Images



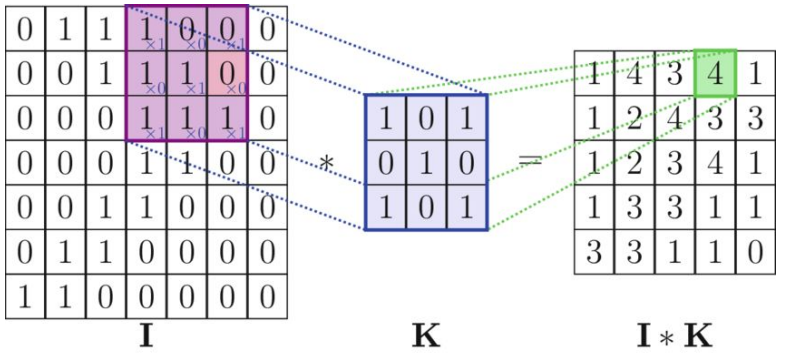
(Feature Extractors)

Learn More (Click)

Components of Traditional CNNs

(1) Convolutional Layers as their name suggest ⇒ Perform mathematical operations called **convolutions**.

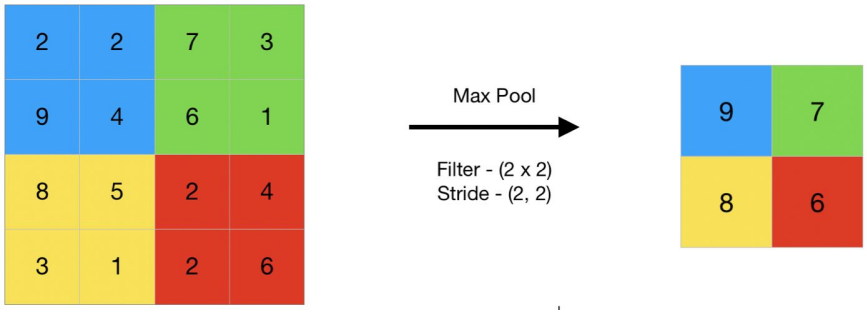
There are **“filters”** which look at small portion of images to **extract meaningful features.**



Components of Traditional CNNs

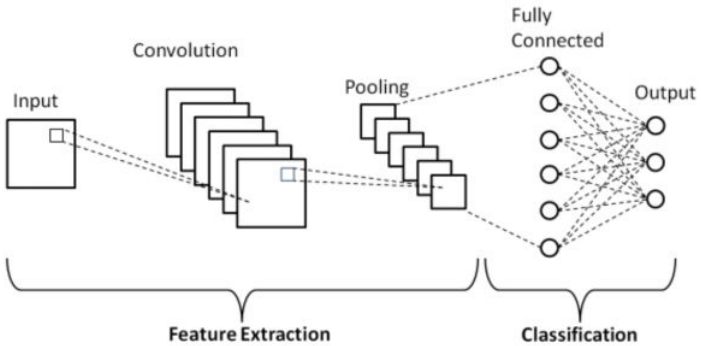
(2) Pooling Layers: Coming after convolutional layers are usually pooling layers.

Besides enhancing computational efficiency, they **combine features to produce more dominant ones.**



Components of Traditional CNNs

(3) ANN Layers: After extracting features, we can flatten them out linearly and **input to ordinary ANNs for classification.**



Implementing CNNs in TensorFlow

Many Hyperparameters to Explore!

```
from tensorflow.keras import layers, models

model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

How Could You Custom CNNs?

- The **first convolutional layer** usually involves "`input_shape`", which is the image size you are working with (*H x W x C*).
- All layers have **activation functions** just like normal ANNs.
- The first number in the layer is the **output's *C***. The size *H* and *W* will be determined from the **filter's size** (e.g. `(3, 3)` or `(2, 2)`).
- As you may already notice, using **larger filter will result in smaller *H* and *W***.
- Useful further readings: visit this **link**.

Implementing CNNs in TensorFlow (cont.)

Adding ANNs layers to the top of the model.

```
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))
```

Note: The output of the prior CNNs are 3D Array. We need to do **flattening** with `Flatten()` before adding linear (`Dense`) layes. Notice that we are doing 10-class image classification so the **output size of the final layer is 10.**

Implementing CNNs in TensorFlow (cont.)

Now we can follow the **standard pipelines** to train and test the models.

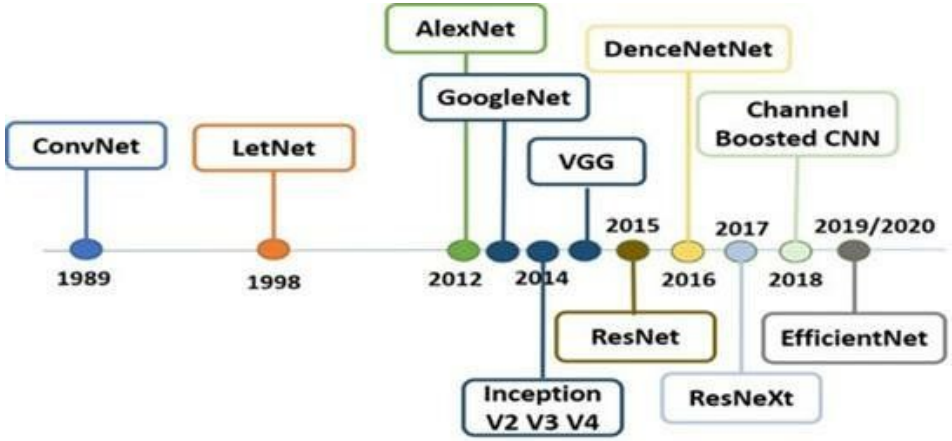
```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
```

Learn More (Click)

Using Other People's CNNs

During the long history, many smart people have already come up with a **really well-performing CNNs**. So, why not just reusing them?



Predeclared CNNs in TensorFlow

In TensorFlow, the famous CNNs are already in the `tf.keras.applications` module. You can find all list **here**.

As you already learned, training **neural networks to provide reliable results** requires **extensive computational resources** as well as **large and high-quality dataset**, which we may usually don't have.

With TensorFlow, we have an option to **reuse knowledge** from the famous CNNs, which are trained on huge dataset and **fine-tune on our tasks!**

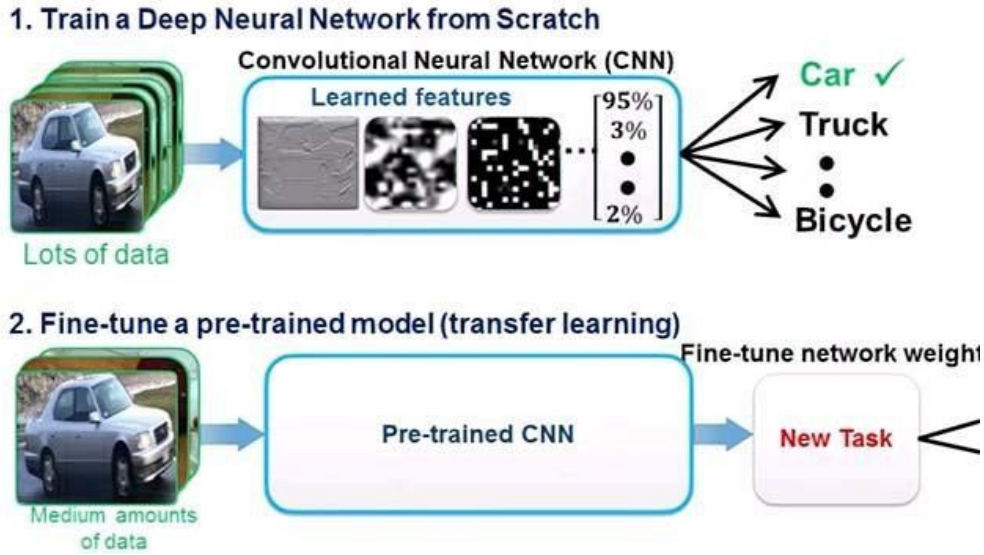
Transfer Learning and Fine-tuning

Transfer Learning is to reuse some of the knowledges from **pre-trained** model and fine-tune on our tasks.

Neural networks normally learn from **random weights**. On the other hand, **pre-trained models provide a great starting point** where models already know how to extract some **low-level features**. This method helps saves some time on training and usually gives better result!

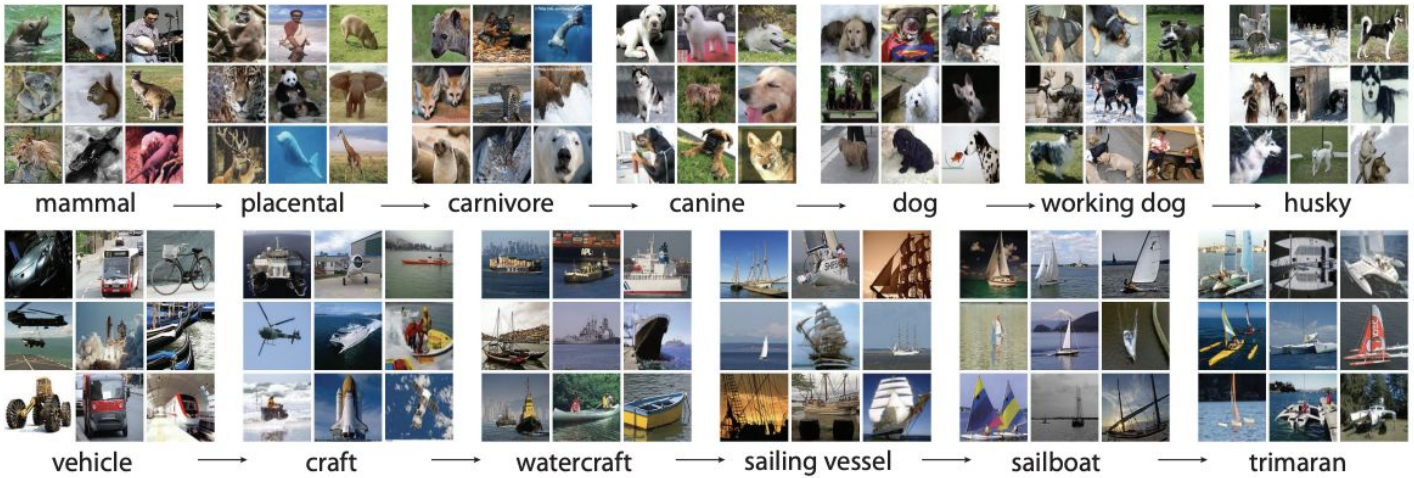
Click Here to Learn More!

Transfer Learning and Fine-tuning (Cont.)



Famous CNNs are Trained on ImageNet

1 Million Images of 1 Thousand Classes



END OF FILE

Let’s Practice What We’ve Just Learned in Action!

[Click Here for Google Colab Notebook!]